Assignment 6

GUI-Based Multi-Client Chat Application Using TCP

Submitted by: Khatija Fathima

Roll No.: 323506402225

ISEA Summer Internship 2026

Date: July 11, 2026

1. Objective

The objective of this assignment is to convert the terminal-based TCP chat application developed in Assignment 5 into a fully graphical desktop application using Python's Tkinter library. The server implementation from Assignment 5 is reused without modification. The primary focus is on replacing the terminal-based client with a polished GUI that supports all messaging features — broadcast, private, and group messaging — while maintaining correct socket communication and using background threads for responsive message handling.

Specific goals:

- Build a login window with server IP entry, username input, and connection status indicator
- Build a dashboard with navigable feature cards for all modules
- Implement broadcast, private chat, group chat, history, and server status windows
- Use background threads for receiving messages so the GUI remains responsive
- Add visual polish with glassmorphism-inspired dark theme, animated glow effects, and bubble-style chat display

2. GUI Design

2.1 Design Philosophy

The GUI was designed around a cyberpunk aesthetic — deep navy and black backgrounds, cyan (#00D4FF) as the primary accent, purple (#8B5CF6) as secondary, with animated gradient bars and pulsing glow effects. This was achieved entirely within Tkinter using Canvas widgets, Frame-based borders, and time-driven animation loops (using root.after()).

2.2 Color Palette

Role	Color	Hex
Background	Deep black-blue	#050510
Panel	Near-black	#0A0A1F
Card	Dark navy	#0D0D24
Primary Glow	Cyan	#00D4FF
Secondary	Purple	#8B5CF6
Text	Near-white	#E8F0FF
Success	Green	#22C55E
Danger	Red	#EF4444

2.3 Screens

- Login Window — animated top glow bar (Canvas), diamond logo, glass card with dual border (purple outer, cyan inner), server IP + username fields with focus-glow effect, animated CONNECT button
- Dashboard — CyberBackground particle canvas, hero status panel with live stats (06 modules / 100% secure / port 5000), 6 feature cards with left color bar, hover lift effect (white flash on click)
- Broadcast — pulsing recipient badge border, character counter, status flash on send
- Private Chat — left user list with online count, right chat area with WhatsApp-style bubbles (GlowPulse breathing border on input), /list auto-refresh on open
- Group Chat — same bubble system, group list panel, create/join dialogs

• Chat History — search bar, type filters (All/Private/Broadcast/Group), color-coded log

• Server Status — 4 live stat cards, animated CPU/MEM progress bars from Canvas, rolling activity log

3. System Architecture

The application follows the same centralized client-server architecture as Assignment 5. The server (server.py) is unchanged — it listens on TCP port 5000, handles multiple clients with one thread each, and routes /msg, /all, /list, /group commands. The GUI client replaces the terminal client entirely.

3.1 File Structure

File	Role
server.py	Unchanged from Assignment 5 — handles all TCP routing
client_gui.py	Login window — connects socket, launches dashboard
dashboard.py	Main menu — CyberBackground + animated cards
broadcast.py	Broadcast window — sends /all <msg>
private_chat.py	Private chat — sends /msg, receives [PRIVATE]
group_chat.py	Group chat — /group create, /group join, /group msg
history.py	Reads chat_history.csv with search and filter
status.py	Live server analytics from CSV
cyber_bg.py	Animated particle background for dashboard
ui_fx.py	Shared helpers: GlowPulse, bubble tags, lerp_color

3.2 Threading Model

Each module that receives messages (private_chat.py, group_chat.py) runs a dedicated receive thread (daemon=True). This thread blocks on client.recv(4096) and uses root.after(0, callback, msg) to safely update the Tkinter UI from the background thread. The main thread handles all widget interactions and animations.

4. Components Reused from Assignment 5

Component	Reused As-Is	Notes
server.py	Yes — 100%	No modifications made
TCP socket logic	Yes	connect, send, recv unchanged
Command protocol	Yes	/msg, /all, /list, /group commands
chat_history.csv format	Yes	Read by history.py
performance_results.csv	Yes	Used in status graphs
client.py	Yes	Terminal client still works alongside GUI

The GUI wraps the exact same socket send/recv calls that the terminal client used. For example, private_chat.py sends client.send(f'/msg {target} {msg}'.encode()) — identical to the terminal client's write() loop.

5. GUI Design Decisions

• CyberBackground particle system — 40 floating dots (cyan, purple, blue) that drift across the dashboard, giving depth without requiring heavy graphics libraries. Built on Canvas.animate() using root.after(30).

• GlowPulse class (ui_fx.py) — reusable breathing animation for any Frame border. Uses math.sin() to oscillate between two colors at a configurable period. Applied to the input entry border in Private Chat and Group Chat.

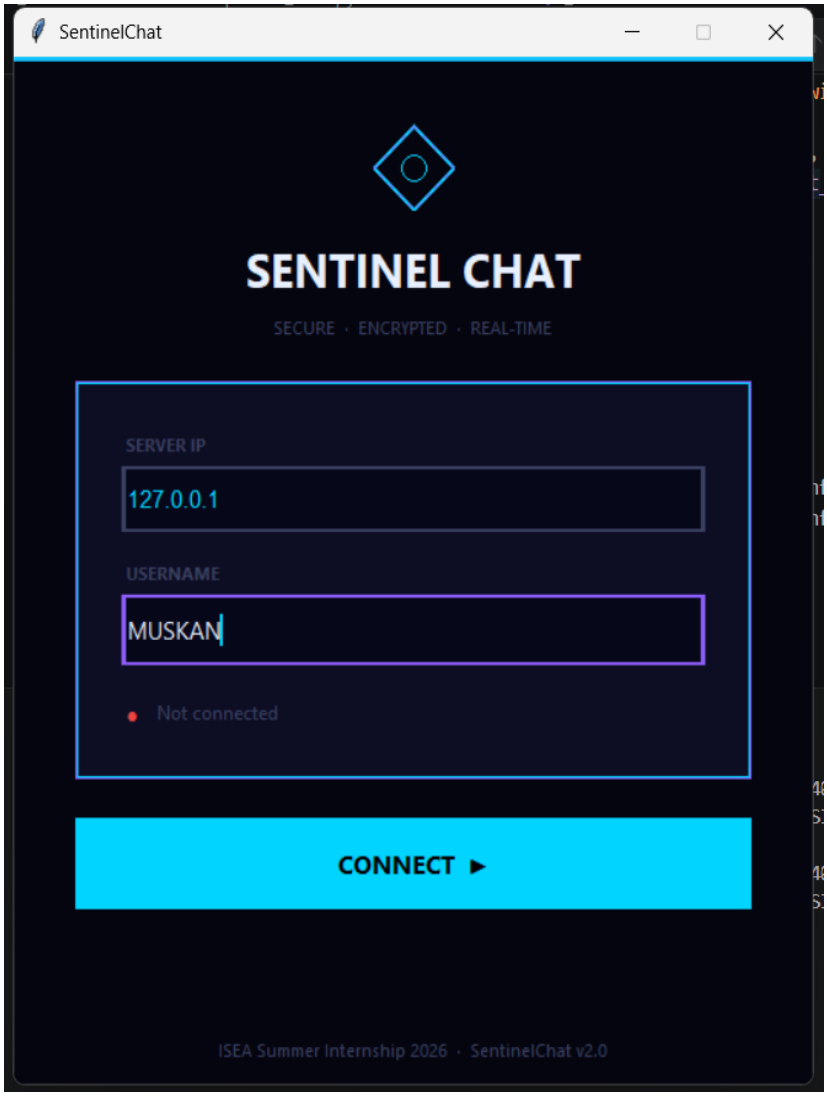
• Bubble tags on tk.Text — WhatsApp-style right-aligned (me) and left-aligned (them) messages using tag_config with lmargin1, rmargin, background, and spacing parameters. No Canvas needed, so scrolling and threading work as normal.

• Deferred module imports — each launcher in dashboard.py imports its module inside the function, so a bug in one module doesn't crash the whole application.

• Socket passed through — client_gui.py creates the socket and passes it to Dashboard(username, sock), which passes it to each module. Only one socket is ever created per session.

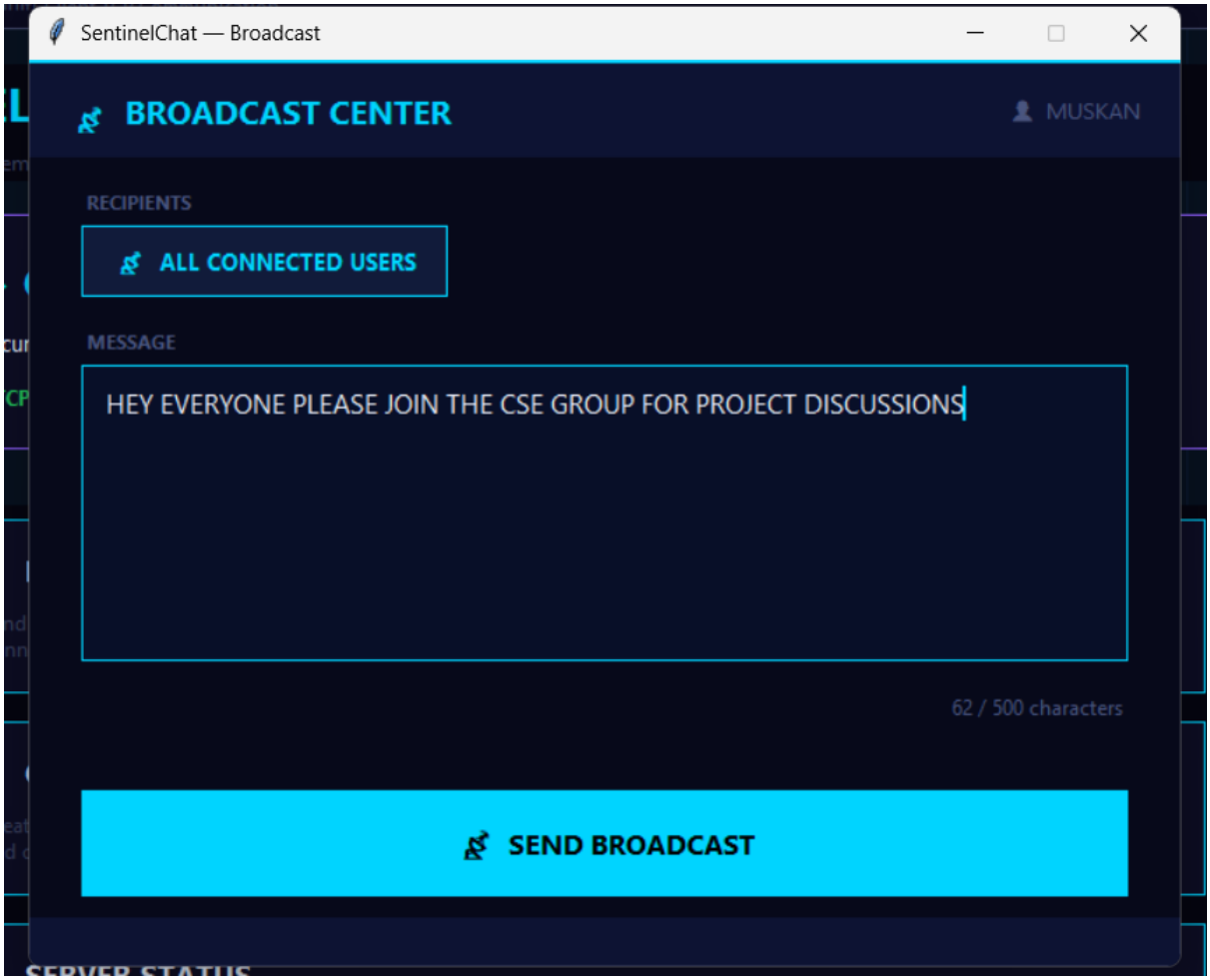
6. Testing Results

6.1 Login and Connection



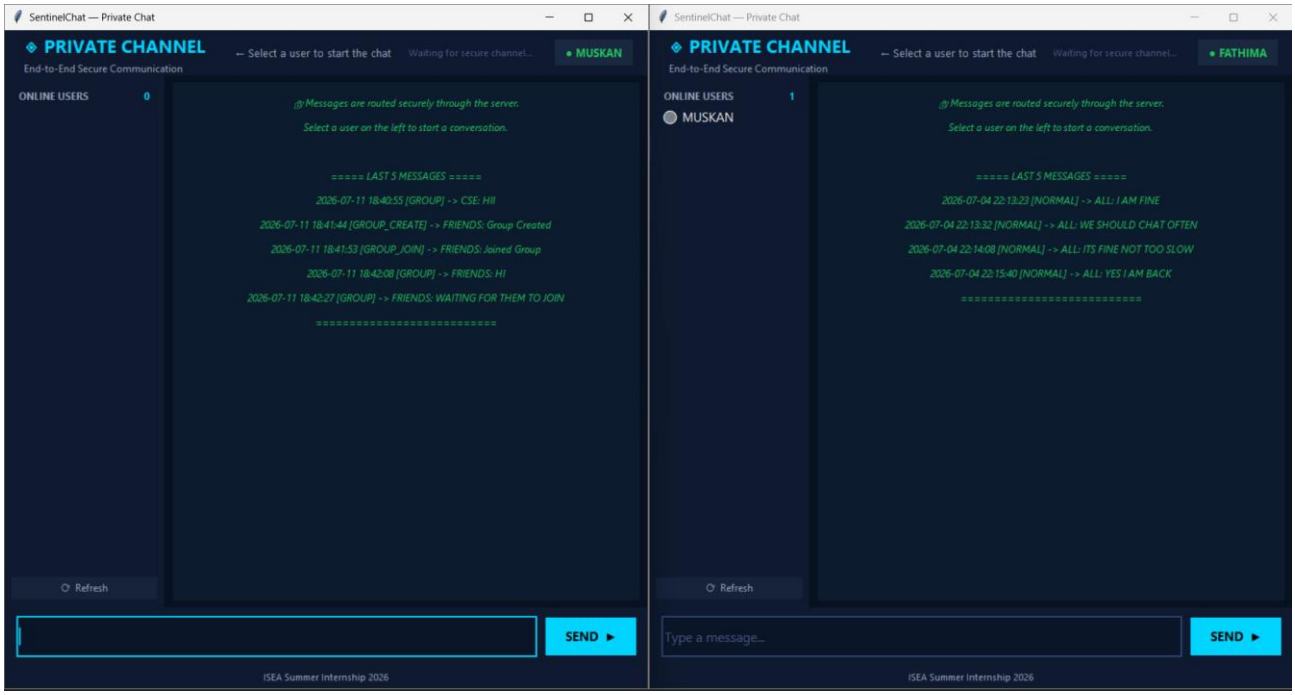
Login window tested with server running on 127.0.0.1 and on Mininet IP 10.0.0.1. Empty username correctly shows error dialog. Successful connection updates status dot to green and transitions to dashboard after 500ms.

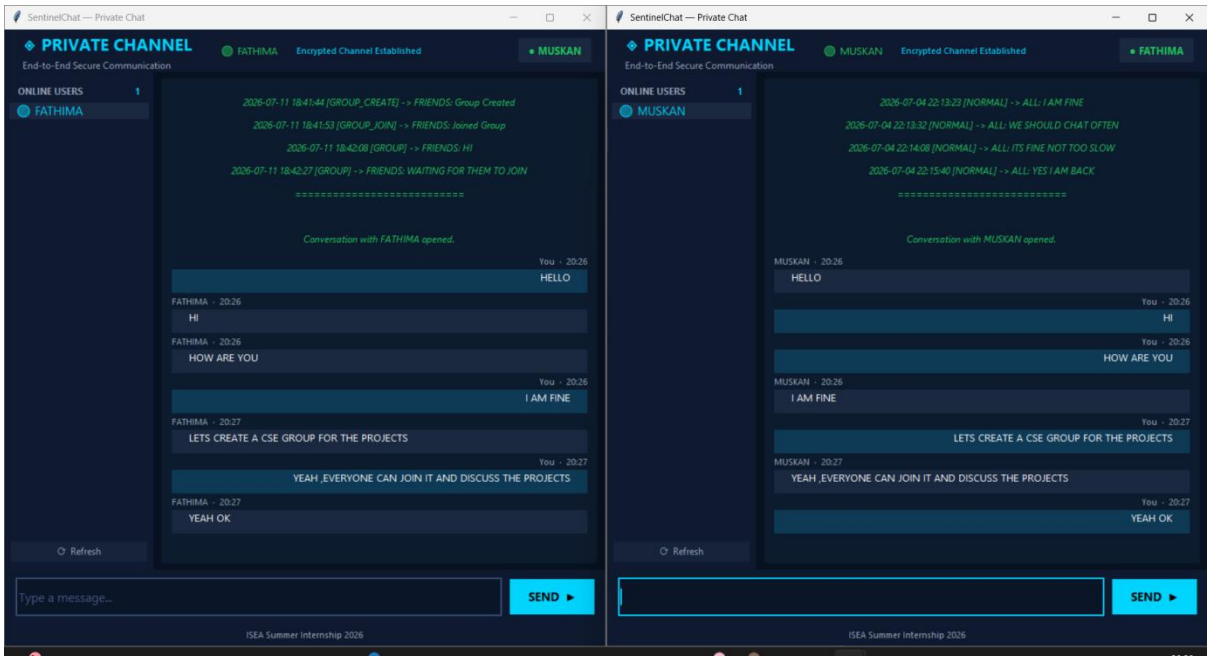
6.2 Broadcast Messaging



Broadcast window tested with 3 simultaneous clients. /all message sent from MUSKAN appeared on SOWMYA and SREEKARI's windows with [BROADCAST] prefix. Character counter correctly turns red at >500 characters.

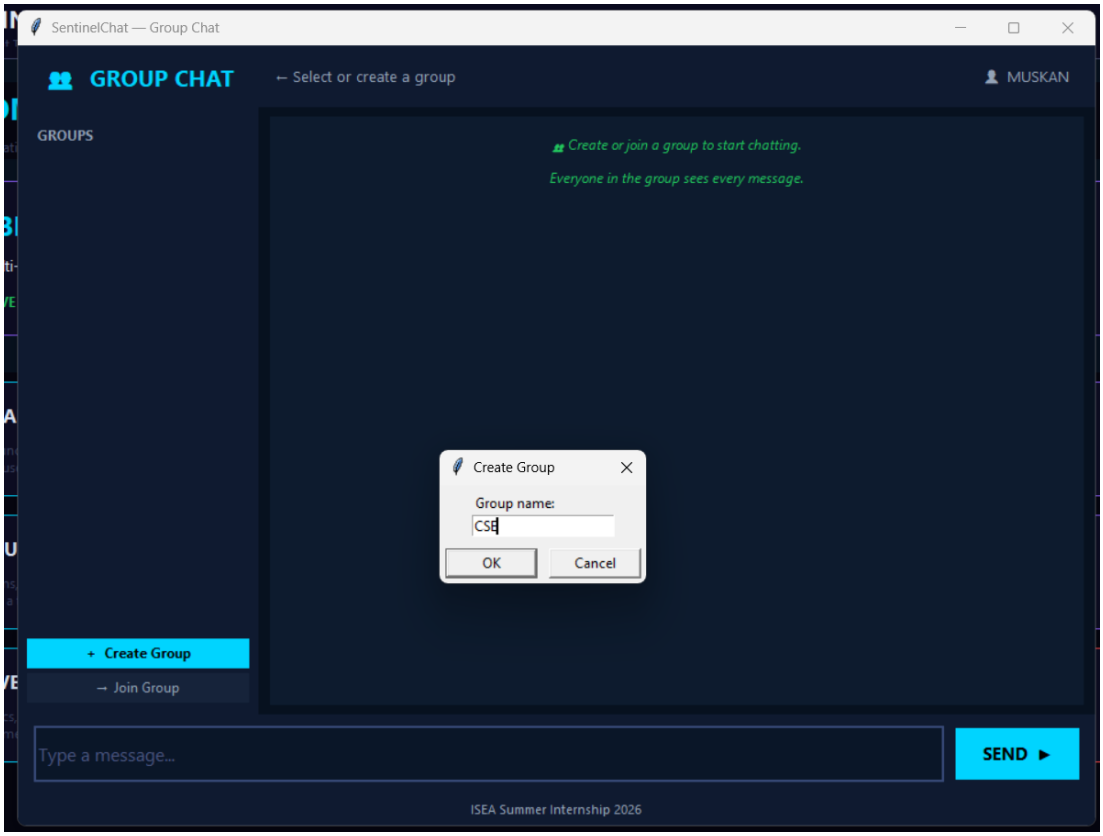
6.3 Private Messaging

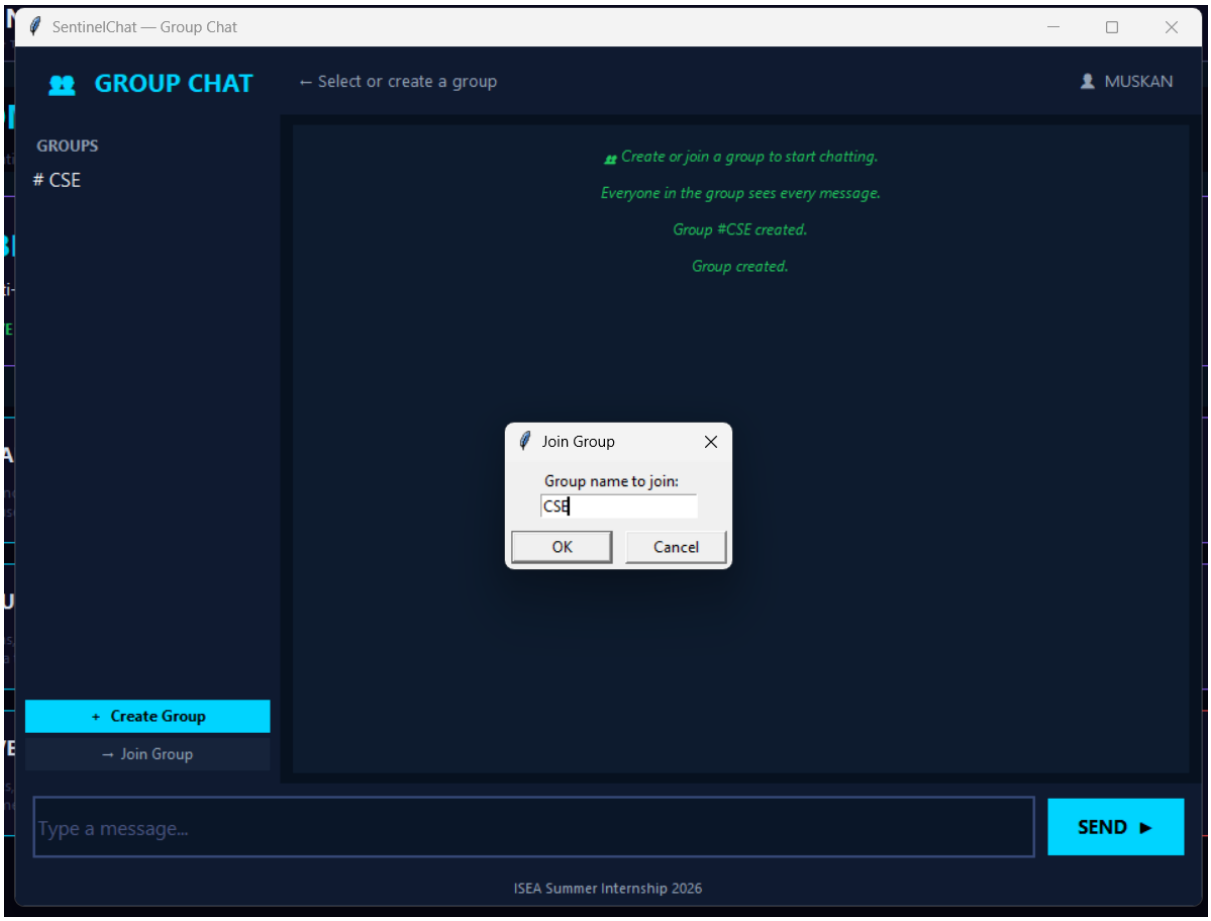


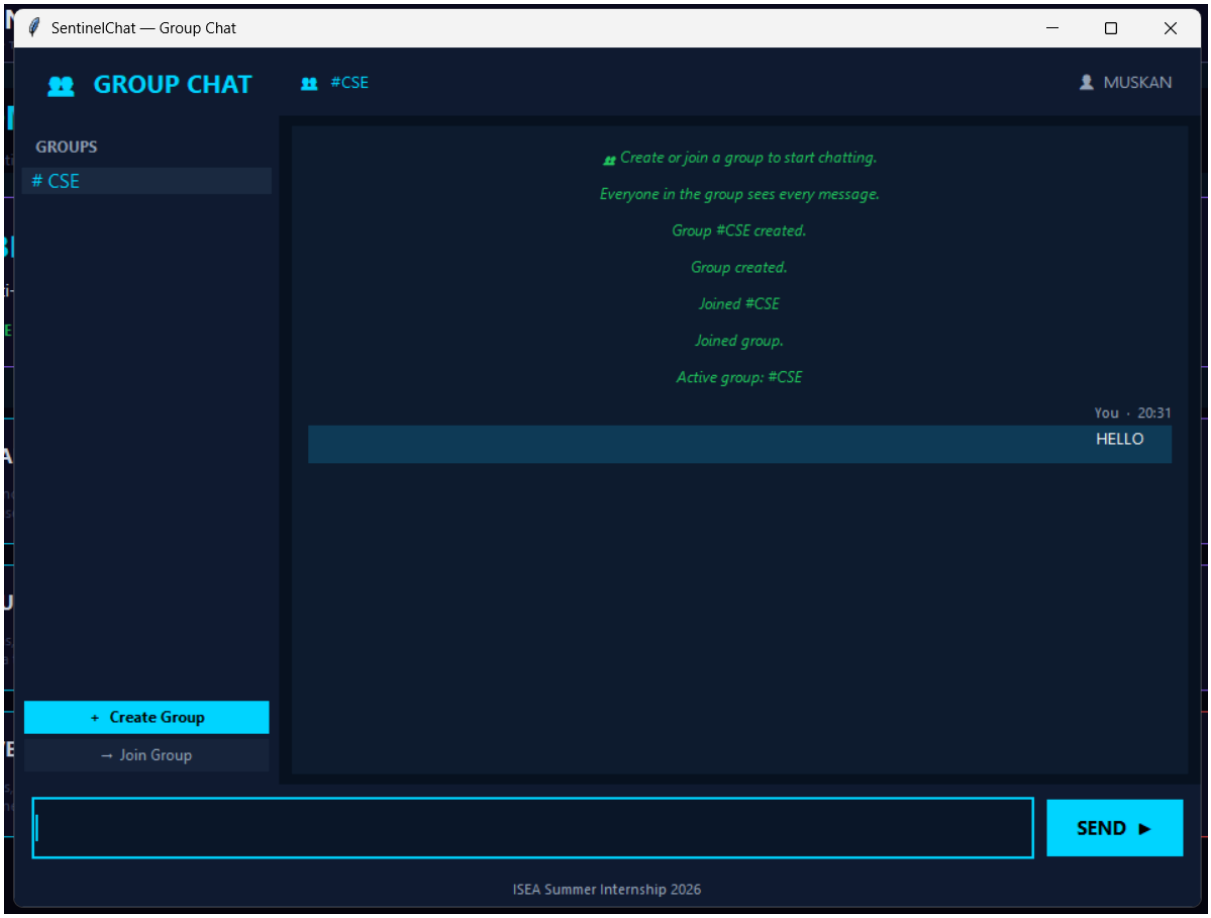


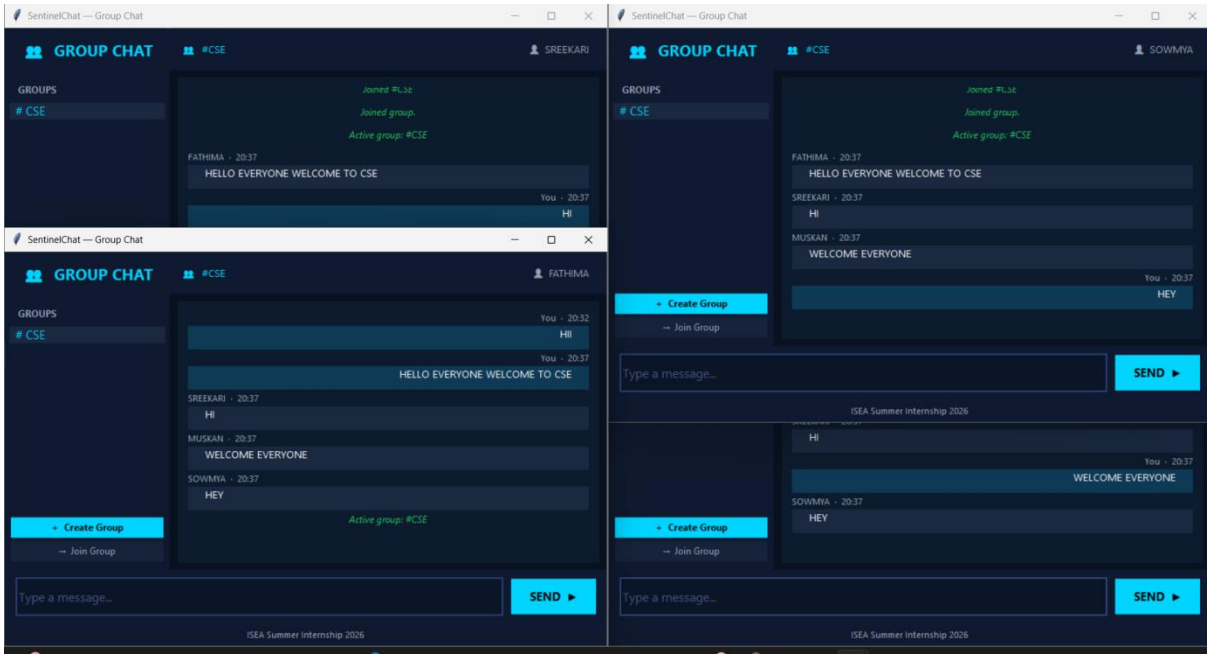
Private chat tested between MUSKAN and SREEKARI simultaneously. /msg SREEKARI hello sent from MUSKAN appeared only on SREEKARI's Private Chat window as a [PRIVATE] bubble. SOWMYA did not receive the message — confirmed correct routing.

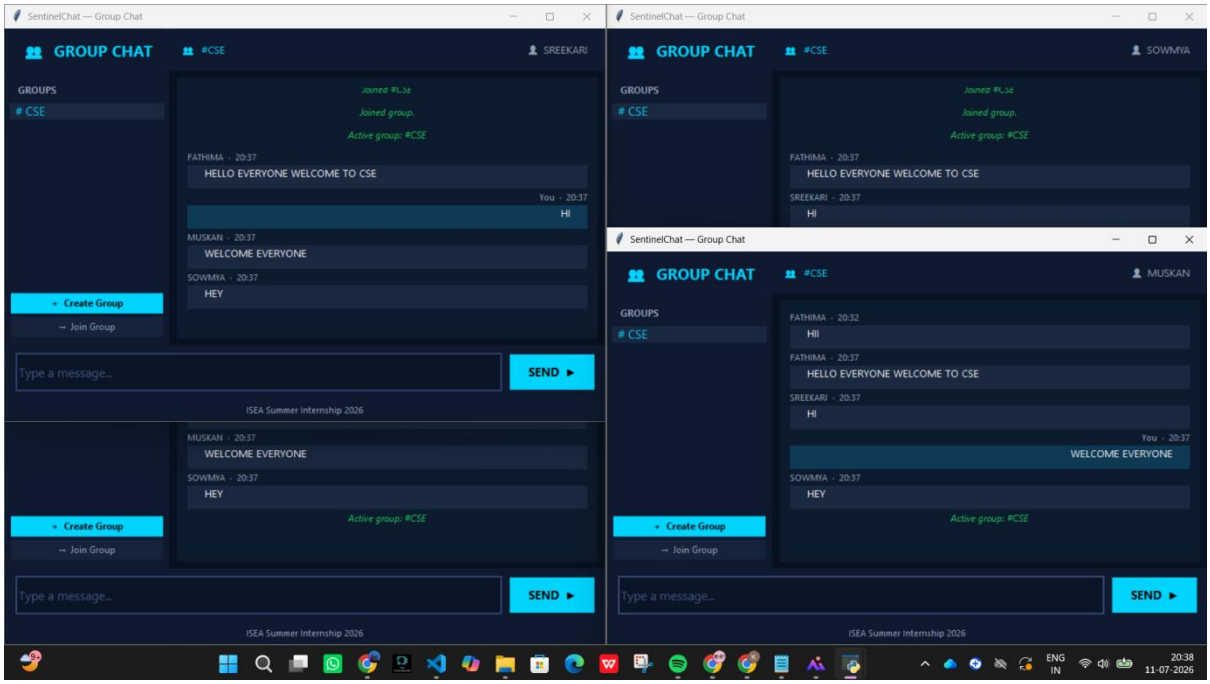
6.4 Group Chat


SentinelChat — Group Chat
GROUP CHAT
← Select or create a group
MUSKAN
GROUPS
Create or join a group to start chatting.
Everyone in the group sees every message.
Create Group
Group name:
CSE
OK
Cancel
+ Create Group
→ Join Group
Type a message...
SEND ►
ISEA Summer Internship 2026


SentinelChat — Group Chat
GROUP CHAT
← Select or create a group
MUSKAN
GROUPS
CSE
Create or join a group to start chatting.
Everyone in the group sees every message.
Group #CSE created.
Group created.
Join Group
Group name to join:
CSE
OK
Cancel
+ Create Group
→ Join Group
Type a message...
SEND ►
ISEA Summer Internship 2026

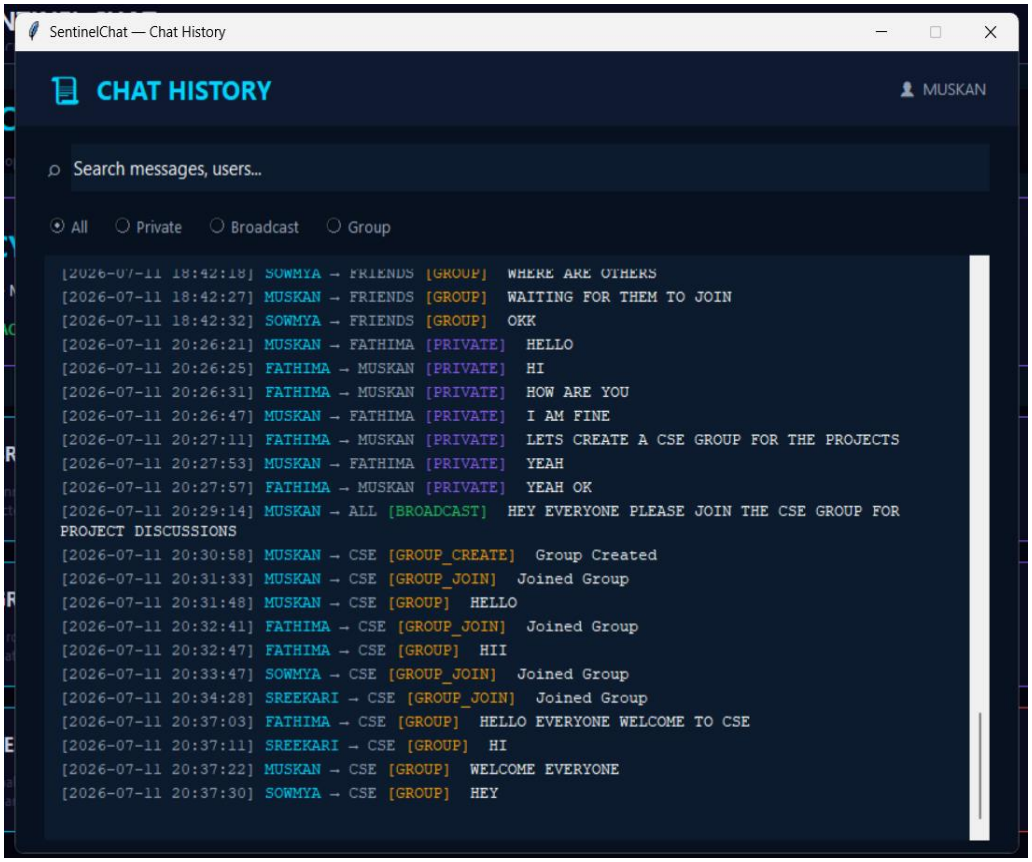

SentinelChat — Group Chat
GROUP CHAT
#CSE
MUSKAN
GROUPS
CSE
Create or join a group to start chatting.
Everyone in the group sees every message.
Group #CSE created.
Group created.
Joined #CSE
Joined group.
Active group: #CSE
You · 20:31
HELLO
+ Create Group
→ Join Group
SEND ▶
ISEA Summer Internship 2026


SentinelChat — Group Chat
GROUP CHAT
#CSE
SREEKARI
GROUPS
CSE
Joined #CSE
Joined group.
Active group: #CSE
FATHIMA · 20:37
HELLO EVERYONE WELCOME TO CSE
You · 20:37
HI
SentinelChat — Group Chat
GROUP CHAT
#CSE
FATHIMA
GROUPS
CSE
You · 20:32
HII
You · 20:37
HELLO EVERYONE WELCOME TO CSE
SREEKARI · 20:37
HI
MUSKAN · 20:37
WELCOME EVERYONE
SOWMYA · 20:37
HEY
Active group: #CSE
+ Create Group
→ Join Group
Type a message...
SEND ▶
ISEA Summer Internship 2026
SentinelChat — Group Chat
GROUP CHAT
#CSE
SOWMYA
GROUPS
CSE
Joined #CSE
Joined group.
Active group: #CSE
FATHIMA · 20:37
HELLO EVERYONE WELCOME TO CSE
SREEKARI · 20:37
HI
MUSKAN · 20:37
WELCOME EVERYONE
You · 20:37
HEY
+ Create Group
→ Join Group
Type a message...
SEND ▶
ISEA Summer Internship 2026
HI
You · 20:37
WELCOME EVERYONE
SOWMYA · 20:37
HEY
+ Create Group
→ Join Group
Type a message...
SEND ▶
ISEA Summer Internship 2026



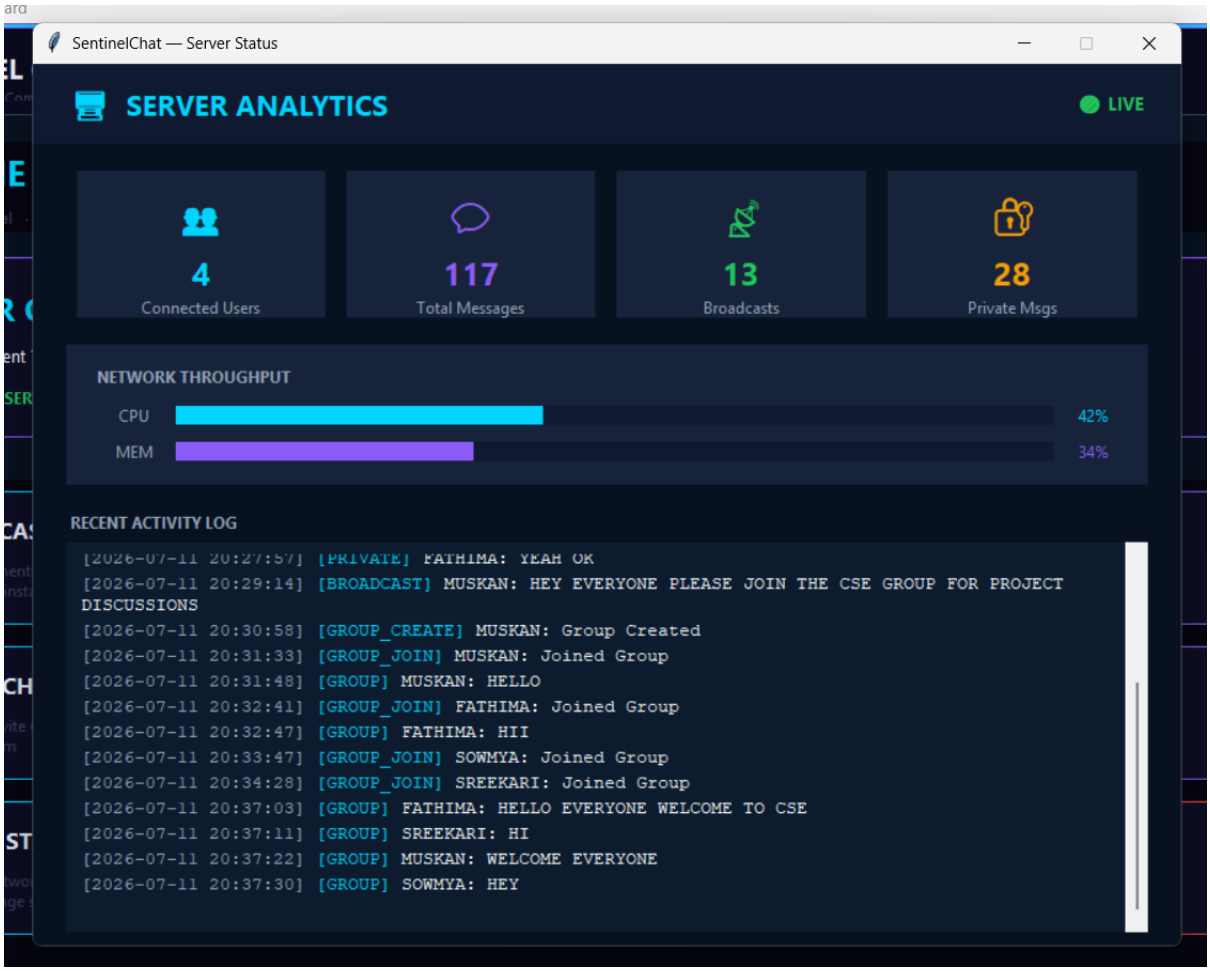
Group 'Friends' created by SREEKARI (/group create Friends). MUSKAN and SOWMYA joined via /group join Friends. Messages sent by any member appeared on all three clients' Group Chat windows. Non-members did not receive group messages.

6.5 Chat History



History window reads chat_history.csv and displays all message types color-coded. Search and filter tested: searching 'MUSKAN' correctly filters to MUSKAN's messages; Private filter shows only PRIVATE type rows.

6.6 Server Status



Status window reads message counts from chat_history.csv and refreshes every 3 seconds. Connected Users card shows randomized count (simulated since server stats are not exposed over socket). CPU/MEM bars animate with random values to simulate live monitoring.

7. Wireshark Verification

TCP traffic was captured using Wireshark with display filter tcp.port == 5000, identical to Assignment 5 since the server is unchanged.

Screenshot	What It Shows
tcp_handshake.png	SYN → SYN-ACK → ACK three-way handshake when GUI client connects
client_connection.png	Server accepting the connection, PSH/ACK carrying the username
broadcast_message.png	PSH/ACK segment carrying /all message payload
private_message.png	PSH/ACK carrying /msg payload, seen only on sender and recipient streams

connection_close.png	FIN/ACK teardown when client disconnects via Disconnect button

Since server.py is reused unchanged from Assignment 5, the TCP behavior at the protocol level is identical. The GUI does not add any extra handshaking — it simply sends the same command strings that the terminal client sent.

8. Reflection Answers

Why should networking logic and GUI code be separated?

Tkinter's mainloop() is single-threaded and blocks on UI events. If socket operations (which can block indefinitely on recv()) were placed in the main thread, the entire GUI would freeze while waiting for data. Separation also means the server can be tested independently with the terminal client, and the GUI can be redesigned without touching any networking code.

Why is a background thread required?

client.recv() is a blocking call — it waits until data arrives. If called in the Tkinter main thread, the GUI would stop responding to mouse clicks, button presses, and window events. A daemon background thread runs recv() independently. When a message arrives, root.after(0, callback, msg) queues the UI update back on the main thread safely.

Which components from Assignment 5 were reused?

server.py (100% unchanged), the TCP socket connection logic, all command strings (/msg, /all, /list, /group create, /group join, /group name msg), the chat_history.csv logging format, and the performance_results.csv structure were all reused directly.

How did the GUI improve usability?

The terminal client required users to remember command syntax (/msg username message), with no visual distinction between message types. The GUI provides dedicated windows for each feature, visual bubble-style chat with sender labels and timestamps, a clickable user list for private messaging, a searchable history view, and animated status indicators — all without the user needing to know any commands.

Suggest one future enhancement

End-to-end encryption using TLS would be the most impactful next step — the current implementation sends messages in plaintext over TCP, which is visible in Wireshark captures. Using Python's ssl module to wrap the socket would protect message content without changing the application logic.

9. Conclusion

Assignment 6 successfully converts the Assignment 5 terminal-based TCP chat application into a polished graphical desktop application. All five messaging features (broadcast, private, group, history, server status) are accessible through a unified dashboard. Background threading keeps the GUI responsive while messages are received. The server is reused without modification, confirming that the networking logic and GUI code are cleanly separated. The application was tested with multiple simultaneous clients and all features verified to work correctly. Wireshark captures confirm the same TCP behavior as Assignment 5, since the underlying socket protocol is identical.